

Tree Tensor Networks as Global and Local Surrogates for Deep Neural Network Training Dynamics

Mathematical construction and a staged empirical program

Hierarchical Feature Learning Project · 2 October 2026

Abstract

This note develops an empirical program for testing whether tree tensor networks can serve as useful surrogates for the training dynamics of deep neural networks. The central distinction is between a *local algebraic surrogate*, obtained by Taylorizing the activations of a smooth neural network near a checkpoint, and a *global autonomous surrogate*, which is trained independently over an entire trajectory. A layerwise degree- k Taylor replacement produces a polynomial network of input degree at most k^L . After homogenization and unrolling, this polynomial network has an exact representation as a constrained, weight-tied, k -ary tree tensor network evaluated on repeated copies of the augmented input. This exact identity does not imply that a freely trained low-rank TTN has the same optimization dynamics, that the representation is compressed, or that singular modes of the unrolled repeated-input tree are meaningful task features.

We therefore propose two complementary experimental tracks. The local track compares a smooth DNN with its own checkpoint-dependent polynomial TTN proxy under identical parameters, data, optimizer state, and update schedule. The global track compares independently trained DNN and TTN students on common teachers, beginning with exactly realizable hierarchical polynomial teachers and progressing through residual TTN teachers, smooth DNN teachers, structured synthetic images, and real image data. Across both tracks, the primary observables are loss, function-space distance, parameter-to-function Jacobians and tangent kernels, teacher-feature coefficients, physical-cut hierarchical singular values and subspaces, acquisition times, and responses to controlled interventions. The experiments are organized to isolate five possible failure points: activation approximation, parameterization geometry, core tying, rank compression, and mismatch between the tensorization and the data distribution.

Central claim to test. A polynomialized DNN always admits an exact constrained TTN representation, but the useful scientific hypothesis is stronger: a substantially simpler TTN should predict nontrivial portions of the DNN’s loss, feature, and intervention trajectories. Algebraic representability, dynamical fidelity, compressibility, and interpretability are separate empirical questions.

Contents

1	Motivation and scope	3
1.1	Research questions	3
2	Mathematical setup	4
2.1	Data, loss, and training trajectories	4
2.2	A general tree tensor network	5
2.3	Smooth feed-forward neural network	5

3	From a smooth DNN to a polynomial tensor network	5
3.1	Layerwise Taylorization	6
3.2	One polynomial layer as a tensor	7
3.3	Homogenization and the exact tied-tree representation	7
3.4	Which tensor representation is most relevant?	8
4	What does dynamical equivalence mean?	8
4.1	Algebraic equivalence is not free-core gradient equivalence	8
4.2	Local prediction and Jacobian defects	9
4.3	Function-space dynamics and the tangent kernel	9
5	Surrogate hypotheses and experimental hierarchy	10
6	Teacher and student families	10
6.1	T0: exact polynomial control	10
6.2	T1: staircase and balanced TTN teachers	11
6.3	T2: residual TTN teacher for soft hierarchy	11
6.4	T3: smooth DNN teacher	11
6.5	T4: structured synthetic images and real images	12
6.6	Student arms	12
6.7	What does “matched capacity” mean?	12
7	Primary observables	12
7.1	Loss and function-space agreement	13
7.2	Parameter-to-function map and tangent kernel	13
7.3	Named teacher-feature coefficients and acquisition times	13
7.4	Physical-cut hierarchical SVD	13
7.5	TTN-specific compression observables	14
7.6	Intervention response	14
8	Experimental protocols	14
8.1	E0: algebra, gradients, and sharing	14
8.2	E1: checkpoint approximation and scaling laws	15
8.3	E2: blind local rollouts	15
8.4	E3: chart stitching and refresh cost	15
8.5	E4: global TTN versus DNN on a TTN teacher	16
8.6	E5: relaxation path from the exact TTN to a free model	16
8.7	E6: DNN teacher	16
8.8	E7: structured images, MNIST, and beyond	17
8.9	E8: causal and negative controls	17
9	Concrete first study	17
9.1	Minimal local-forecast algorithm	18
9.2	Minimal global-comparison algorithm	18

10 Statistical analysis and decision criteria	19
10.1 Primary endpoints	19
10.2 Paired design and uncertainty	19
10.3 Success criteria	19
11 Failure modes and breakdown diagnostics	20
12 Expected outcomes and theory roadmap	20
13 Reproducibility checklist	21
14 Conclusion	21

1 Motivation and scope

Deep neural networks learn through a high-dimensional, strongly coupled parameter trajectory. Even when the target task has a simple compositional structure, it is usually difficult to identify a small set of variables whose dynamics explain when particular features appear, how features gate one another, or why an intervention at one level changes learning at another. Tree tensor networks are attractive candidate toy models because they make compositional structure, intermediate ranks, and cut-wise feature spaces explicit. They may therefore support reduced descriptions in terms of coefficient modes, hierarchical singular values, subspaces, and a small number of coupling variables.

The proposal has two logically distinct motivations.

Global model of training. A single TTN architecture is trained from initialization to convergence alongside a DNN on the same teacher and data. The hypothesis is not that the two parameter vectors coincide, but that their trajectories agree in function space and in selected feature observables. A global TTN would be especially valuable if it is substantially smaller and if its internal variables admit tractable dynamical equations.

Local model of training. At selected DNN checkpoints, each smooth activation is replaced by a low-order polynomial around a specified center. The resulting polynomial network can be written exactly as a constrained TTN. The local question is whether this proxy preserves the DNN gradient field and feature evolution over a nontrivial forecast window. A sequence of such local charts could provide an interpretable “atlas” of training even when no one TTN is faithful globally.

The two tracks answer different questions. The local track can begin from identical parameters and can compare gradients directly. The global track is more ambitious but also more parameterization-dependent: two models can have comparable function classes and still move at different speeds or along different paths because their parameter-to-function maps differ.

1.1 Research questions

The empirical program is organized around five questions.

1. **Local vector-field fidelity.** Over what horizon does a low-order polynomial TTN predict the continuation of the same smooth DNN from a checkpoint?
2. **Global phenomenological fidelity.** When a TTN and a DNN are trained independently on the same teacher, do they acquire the same task features in the same order and respond similarly to interventions?

3. **Geometric fidelity.** Are similarities or differences explained by the parameter-to-function Jacobian and the induced tangent kernel rather than by representational capacity alone?
4. **Compressibility.** Can the exact, often large, polynomial representation be reduced to low physical TTN ranks without losing predictions or gradients?
5. **Breakdown.** How does fidelity deteriorate with depth, activation spread, stochastic optimization, residual pathways, correlated data, and realistic images?

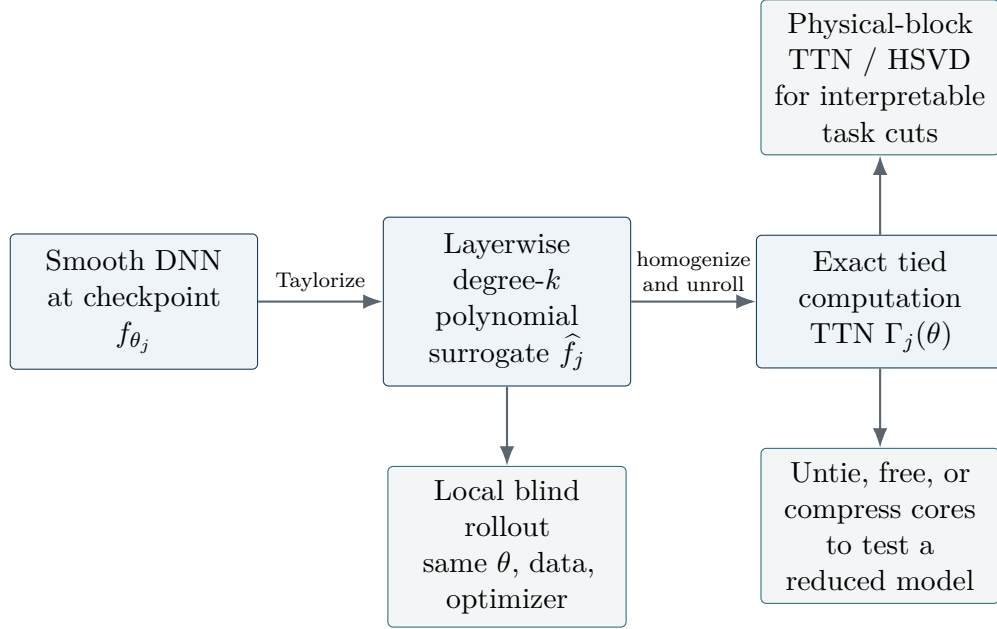


Figure 1: The proposed surrogate ladder. The exact TTN is an algebraic rendering of the polynomial network. Local forecasting tests the Taylor approximation. Relaxation and compression test whether a simpler autonomous tensor model retains the relevant dynamics.

2 Mathematical setup

2.1 Data, loss, and training trajectories

Let $(x, y) \sim \nu$ with $x \in \mathbb{R}^{d_0}$ and scalar y . The initial theory and synthetic experiments use population or full-batch square loss,

$$\mathcal{L}(f) = \frac{1}{2} \mathbb{E}_{\nu} \left[(f(x) - y)^2 \right]. \quad (1)$$

For a parameterized model f_{ϑ} , gradient flow is

$$\dot{\vartheta}(t) = -\nabla_{\vartheta} \mathcal{L}(f_{\vartheta}), \quad (2)$$

with the direct discrete analogue for gradient descent. Later stages may use stochastic gradients and cross-entropy, but square loss is the cleanest starting point because prediction, Jacobian, and feature errors enter the gradient transparently.

We write

$$\Phi_{\mathbf{M}} : \vartheta \longmapsto f_{\vartheta} \in L^2(\nu_x) \quad (3)$$

for the parameter-to-function map of model class $\mathbf{M}$. Its differential $D\Phi_{\mathbf{M}}(\vartheta)$ and the associated

tangent kernel summarize how parameter motion changes the learned function. They are central observables whenever two models use different parameterizations.

2.2 A general tree tensor network

Let $\mathcal{T} = (V, E)$ be a rooted tree with leaf set $\mathcal{L}$. Each leaf $i \in \mathcal{L}$ receives a block $x^{(i)} \in \mathcal{X}_i$ and a feature map

$$\phi_i : \mathcal{X}_i \rightarrow \mathbb{R}^{r_i}. \quad (4)$$

Each internal node v has children $\text{ch}(v) = \{u_1, \dots, u_{m_v}\}$ and a multilinear core

$$A_v : \mathbb{R}^{r_{u_1}} \times \dots \times \mathbb{R}^{r_{u_{m_v}}} \rightarrow \mathbb{R}^{r_v}. \quad (5)$$

The node states are defined recursively by

$$h_i = \phi_i(x^{(i)}), \quad h_v = A_v(h_{u_1}, \dots, h_{u_{m_v}}). \quad (6)$$

At the root r , either $r_r = 1$ or a final readout $c \in \mathbb{R}^{r_r}$ gives

$$g_A(x) = c^\top h_r. \quad (7)$$

The dimensions r_v are the bond dimensions or hierarchical ranks. Contracting all internal indices produces a coefficient tensor. Conversely, any finite coefficient tensor admits a TTN factorization if the bond dimensions are allowed to reach the ranks of the corresponding matricizations. Thus TTNs are universal finite tensor factorizations, but low-rank representability is an additional property, not a theorem for arbitrary polynomials.

A TTN parameterization has gauge freedom: an invertible transformation on an internal bond can be absorbed by the two adjacent cores without changing the represented function. Raw core entries are therefore not suitable primary observables unless a canonical gauge is fixed. Singular values, subspaces, function values, and intervention responses are safer.

2.3 Smooth feed-forward neural network

Let the DNN have L nonlinear hidden layers with widths $d_1, \dots, d_L$:

$$h_0(\theta, x) = x, \quad (8)$$

$$z_\ell(\theta, x) = W_\ell h_{\ell-1}(\theta, x) + b_\ell, \quad (9)$$

$$h_\ell(\theta, x) = \sigma_\ell(z_\ell(\theta, x)), \quad 1 \leq \ell \leq L, \quad (10)$$

$$f_\theta(x) = c^\top h_L(\theta, x) + b_{\text{out}}. \quad (11)$$

Each σ_ℓ acts coordinatewise. For an order- k Taylor construction we assume the scalar activation components are C^{k+1} on intervals containing all preactivations used by the DNN and its proxy. The first experiments exclude dropout, normalization, attention, and nonsmooth activations. Affine residual skips can be added without changing polynomial representability.

3 From a smooth DNN to a polynomial tensor network

3.1 Layerwise Taylorization

Fix a checkpoint index j and centers $m_{\ell,j,i}$ for each neuron. Define the scalar degree- k Taylor polynomial

$$p_{\ell,j,i}^{(k)}(z) = \sum_{r=0}^k \frac{\sigma_{\ell,i}^{(r)}(m_{\ell,j,i})}{r!} (z - m_{\ell,j,i})^r. \quad (12)$$

Replacing every activation by $p_{\ell,j}^{(k)}$ gives the autonomous polynomial recurrence

$$\hat{h}_{0,j}(\theta, x) = x, \quad (13)$$

$$\hat{z}_{\ell,j}(\theta, x) = W_{\ell} \hat{h}_{\ell-1,j}(\theta, x) + b_{\ell}, \quad (14)$$

$$\hat{h}_{\ell,j}(\theta, x) = p_{\ell,j}^{(k)}(\hat{z}_{\ell,j}(\theta, x)), \quad (15)$$

$$\hat{f}_j(\theta, x) = c^{\top} \hat{h}_{L,j}(\theta, x) + b_{\text{out}}. \quad (16)$$

The proxy must use its own hidden states. Substituting the true DNN hidden states at every layer would define an error diagnostic, not a trainable surrogate.

If the centers are independent of x , then $\hat{f}_j(\theta, \cdot)$ is a polynomial in the input. Its maximum degree is k^L , not k , because degrees multiply under composition. Taylor's theorem gives the scalar bounds

$$|\sigma(z) - p^{(k)}(z)| \leq \frac{M_{k+1}}{(k+1)!} |z - m|^{k+1}, \quad (17)$$

$$|\sigma'(z) - (p^{(k)})'(z)| \leq \frac{M_{k+1}}{k!} |z - m|^k, \quad (18)$$

where M_{k+1} bounds the $(k+1)$ st derivative along the relevant segment. The second estimate matters because a small prediction error need not imply a small training-gradient error.

Two notions of locality

Fixed-center input polynomial.

The centers are fixed independently of x , for example neuronwise checkpoint means on a calibration set. This produces one polynomial function of the input and therefore one fixed-core tensor representation at each parameter value. It may already have a nonzero error at the checkpoint because preactivations are spread around the centers.

Example-centered checkpoint expansion.

For each example, choose $m_{\ell,j}(x) = z_{\ell}(\theta_j, x)$ and freeze those reference functions over the local window. This matches the DNN and its parameter derivatives through order k at θ_j under suitable smoothness assumptions. It is a strong temporal control, but its coefficients depend on x , so it is not generally one simple fixed polynomial in the input.

The fixed-center construction is the main TTN object. The example-centered construction is an important diagnostic: if it works while the fixed-center model fails, the problem is activation spread across inputs rather than the tensor identity.

3.2 One polynomial layer as a tensor

For a fixed layer and checkpoint, write

$$P_\ell(h) = p_{\ell,j}^{(k)}(W_\ell h + b_\ell) = \sum_{r=0}^k A_\ell^{(r)}[h^{\otimes r}], \quad (19)$$

where $A_\ell^{(r)}$ is a symmetric r -linear map from $(\mathbb{R}^{d_\ell-1})^r$ to $\mathbb{R}^{d_\ell}$. If $w_{\ell,i}^\top$ is row i of W_ℓ and $q_{\ell,i} = b_{\ell,i} - m_{\ell,j,i}$, then

$$A_{\ell,i}^{(r)} = \left[\sum_{s=r}^k \frac{\sigma_{\ell,i}^{(s)}(m_{\ell,j,i})}{r!(s-r)!} q_{\ell,i}^{s-r} \right] w_{\ell,i}^{\otimes r}. \quad (20)$$

This formula exposes an important constraint: the tensors at different polynomial orders reuse the same neural weight row. They are not independent dense trainable tensors.

3.3 Homogenization and the exact tied-tree representation

Introduce the augmented space $H_\ell = \mathbb{R} \oplus \mathbb{R}^{d_\ell}$ and augmented state $\bar{h} = (1, h)$. Define a symmetric k -linear map

$$G_\ell : H_{\ell-1}^{\otimes k} \rightarrow H_\ell \quad (21)$$

by, for $u_s = (a_s, v_s)$,

$$(G_\ell(u_1, \dots, u_k))_0 = \prod_{s=1}^k a_s, \quad (22)$$

$$(G_\ell(u_1, \dots, u_k))_{1:d_\ell} = \sum_{r=0}^k \binom{k}{r}^{-1} \sum_{\substack{S \subseteq \{1, \dots, k\} \\ |S|=r}} A_\ell^{(r)}[v_s : s \in S] \prod_{s \notin S} a_s. \quad (23)$$

The binomial factor prevents lower-order terms from being counted repeatedly. On repeated arguments,

$$G_\ell(\bar{h}, \dots, \bar{h}) = (1, P_\ell(h)). \quad (24)$$

Proposition 1: exact computation TTN. For $k \geq 2$, the depth- L polynomial network has an exact representation by a rooted k -ary TTN with $N = k^L$ formal leaves. Every node at height ℓ uses the same core G_ℓ , all formal leaves receive the same augmented input $\bar{x} = (1, x)$, and with $\bar{c} = (b_{\text{out}}, c)$,

$$\hat{f}_j(\theta, x) = \langle T_j(\theta), \bar{x}^{\otimes N} \rangle. \quad (25)$$

Every bond carrying a height- ℓ hidden state has dimension at most $d_\ell + 1$.

Proof sketch. The homogenized core reproduces one polynomial layer when all k inputs are equal. Substitute the previous layer into each of these k occurrences and continue recursively. After L substitutions there are k^L formal leaves, while nodes at equal height share the same parameterized core. The equality holds for every parameter value because it is an algebraic identity. $\square$

This proposition answers the representability question affirmatively, but with qualifications:

- The representation is a *constrained, weight-tied TTN*, not a generic TTN with independent core entries.

- It is evaluated on the diagonal input $(\bar{x}, \dots, \bar{x})$; the formal leaves are repeated occurrences of the full input, not disjoint physical variables.
- The exponential number of formal leaves need not be materialized. The original polynomial recurrence evaluates the same function while reusing shared subcomputations.
- Low physical TTN rank does not follow from this construction. It must be measured after tensorizing the actual input variables or blocks.

3.4 Which tensor representation is most relevant?

Three representations should be kept distinct.

Representation	Main advantage	Main limitation
Unrolled computation TTN	Exact identity with the polynomial DNN; preserves layer structure and, if parameterized by θ , preserves polynomial-network gradients.	Repeated full-input leaves; artificial tree cuts; not automatically compressed.
Shared tensor circuit / DAG	Closest to the efficient polynomial computation graph because repeated subcomputations remain shared.	Not a tree; some TTN tools and closed-form rank notions no longer apply directly.
Physical-block TTN from coefficient tensor or HSVD	Cuts correspond to real input blocks; suitable for interpretable singular modes and compression.	Conversion can increase ranks; a truncation changes both the function and generally its Jacobian.

Table 1: Complementary tensor representations of a polynomialized DNN.

The unrolled TTN is the correct object for algebraic and local dynamical equivalence. The physical-block TTN is usually the correct object for feature interpretation. A shared DAG tensor circuit may be the best computational surrogate. No one representation is optimal for all three purposes.

4 What does dynamical equivalence mean?

4.1 Algebraic equivalence is not free-core gradient equivalence

Let $g = \Gamma_j(\theta)$ denote the unique tied cores and readout produced by the polynomial construction, and let $\ell(g)$ be the loss written as a function of those core variables. If the proxy is trained in the original neural parameters,

$$\dot{\theta} = -\nabla_{\theta} \hat{\mathcal{L}}_j(\theta), \quad (26)$$

then the induced core trajectory is

$$\dot{g} = -D\Gamma_j(\theta)D\Gamma_j(\theta)^{\top}\nabla_g\ell(g). \quad (27)$$

Thus Euclidean gradient flow in neural parameters becomes a generally non-Euclidean and possibly degenerate preconditioned flow in core space. Ordinary Euclidean gradient descent on free core entries is a different dynamical system, even when initialized at the same represented function.

This observation motivates a controlled relaxation ladder:

1. train the exact tied TTN through the original parameters θ ;
2. train the same tied cores as free entries using the induced metric in Eq. (27);
3. replace the induced metric by a diagonal or block-diagonal approximation;

4. use ordinary Euclidean core gradients;
5. untie repeated core occurrences;
6. compress to lower physical ranks;
7. change the tree.

The first step should be identical to the polynomial network. Each subsequent step isolates a specific simplification.

4.2 Local prediction and Jacobian defects

At a common parameter value define

$$\epsilon_{0,j}(\theta) = \|f_\theta - \hat{f}_j(\theta)\|_{L^2(\nu_x)}, \quad (28)$$

$$\epsilon_{1,j}(\theta) = \|\nabla_\theta f_\theta - \nabla_\theta \hat{f}_j(\theta)\|_{L^2(\nu_x)}, \quad (29)$$

$$R_y(\theta) = \|f_\theta - y\|_{L^2(\nu)}, \quad (30)$$

$$B_j(\theta) = \|\nabla_\theta \hat{f}_j(\theta)\|_{L^2(\nu_x)}. \quad (31)$$

For square loss,

$$\|\nabla \mathcal{L}(\theta) - \nabla \hat{\mathcal{L}}_j(\theta)\| \leq R_y(\theta)\epsilon_{1,j}(\theta) + B_j(\theta)\epsilon_{0,j}(\theta) =: \beta_j(\theta). \quad (32)$$

The bound shows why prediction agreement alone is insufficient. Two functions can be close while their parameter derivatives, and therefore their training updates, are different.

If the true vector field is K_j -Lipschitz in a common tube and the common-point defect is bounded by β_j , then two gradient-flow trajectories satisfy

$$E(s) \leq e^{K_j s} E(0) + \beta_j \frac{e^{K_j s} - 1}{K_j}, \quad E(s) = \|\theta(t_j + s) - \tilde{\theta}(t_j + s)\|. \quad (33)$$

The fraction is interpreted as s when $K_j = 0$. This is a conditional local statement, not a guarantee that practical constants are small.

4.3 Function-space dynamics and the tangent kernel

For an audit set $X = (x_1, \dots, x_n)$, let

$$F_\theta = (f_\theta(x_1), \dots, f_\theta(x_n))^\top, \quad J_\theta = D_\theta F_\theta, \quad K_\theta = J_\theta J_\theta^\top. \quad (34)$$

Under full-batch square-loss gradient flow,

$$\dot{F}_\theta = -K_\theta(F_\theta - y). \quad (35)$$

For a TTN with parameters φ , define G_φ , J_φ , and K_φ analogously. Their prediction difference evolves as

$$\frac{d}{dt}(F_\theta - G_\varphi) = -K_\theta(F_\theta - G_\varphi) - (K_\theta - K_\varphi)(G_\varphi - y). \quad (36)$$

Equation (36) gives a practical global criterion: function agreement is more plausible when both the predictions and the residual-weighted tangent kernels agree. It also supplies a concrete meaning for comparing the “function-parameter map.” The full Jacobians need not have the same number of columns; their Gram operators on examples can still be compared.

5 Surrogate hypotheses and experimental hierarchy

Model	Parameterization	What it tests	What would count as failure
Exact polynomial recurrence	Original DNN parameters θ ; frozen activation coefficients	Accuracy of activation polynomialization	Large checkpoint prediction/Jacobian defect; very short forecast horizon
Exact tied computation TTN	Same θ mapped through Γ_j	Algebra and implementation	Any disagreement with the polynomial recurrence
Free tied TTN	Core entries trainable; ties retained	Sensitivity to parameterization geometry	Rapid divergence despite identical initial function
Untied TTN	Each occurrence independent	Role of weight sharing	Feature and loss trajectories change when ties are released
Compressed physical TTN	Low HSVD ranks on actual input blocks	Existence of a reduced interpretable model	Small function error but large Jacobian or intervention error
Independent global TTN	Fixed tree and ranks, trained from initialization	Phenomenological global surrogate hypothesis	Different feature order, kernel evolution, or intervention response after capacity matching

Table 2: The surrogate hierarchy separates distinct reasons why a tensor model may succeed or fail.

The principal hypotheses are:

H1: useful local windows. For modest depth and smooth activations, orders $k = 2, 3, 4$ provide nontrivial windows over which the proxy predicts both loss and named feature trajectories. The accepted window should grow with k and shrink with activation spread and depth.

H2: gradient defect predicts horizon. The measured loss-gradient defect in Eq. (32) should predict local rollout error more reliably than checkpoint prediction error alone.

H3: global feature fidelity on hierarchical teachers. After controlling for realizability and training stability, DNN and TTN students should display correlated teacher-feature acquisition times, physical-cut singular modes, and intervention effects even if their raw parameter trajectories differ.

H4: a reduced physical TTN exists in simple regimes. For low-rank hierarchical teachers, the DNN and its polynomial proxies should remain compressible on the teacher-aligned physical dimension tree. The required ranks should grow predictably as the teacher, data, or student becomes less aligned.

H5: breakdown is structured. As experiments become more realistic, failure should be attributable to a measurable component: Taylor remainder, tangent-kernel mismatch, rank growth, correlated-input geometry, optimizer memory, or a wrong tree. A clean failure taxonomy is itself a useful result.

6 Teacher and student families

6.1 T0: exact polynomial control

Use a polynomial activation such as $\sigma(z) = z + \alpha z^2$ in a small depth-two network. Its order-two Taylor replacement is exact for every center. The sequential polynomial network and the tied TTN

must agree globally in values and gradients. This is the strongest implementation control because any discrepancy cannot be blamed on approximation error.

A second exact control uses a planted low-rank TTN teacher and a student with the same TTN tree and sufficient ranks. This checks optimizer and feature-measurement code independently of the DNN-to-TTN construction.

6.2 T1: staircase and balanced TTN teachers

For a simple nested teacher, partition the input into independent Gaussian blocks

$$x = (x^{(1)}, \dots, x^{(M)}), \quad x^{(i)} \sim \mathcal{N}(0, I_b), \quad (37)$$

choose unit vectors u_i , and define

$$z_i = u_i^\top x^{(i)}, \quad \phi_m(x) = \prod_{i=1}^m z_i, \quad f^*(x) = \sum_{m=1}^M \sqrt{\lambda_m} \phi_m(x). \quad (38)$$

The features are orthonormal under the product Gaussian law, so their coefficients can be measured directly. The teacher is hierarchical, but no acquisition order is assumed for the DNN.

A wider branching teacher uses a binary tree over the blocks. At leaf i ,

$$h_i^* = U_i x^{(i)} \in \mathbb{R}^{r_i}, \quad (39)$$

and at an internal node v ,

$$h_v^* = B_v(h_{v_L}^*, h_{v_R}^*), \quad (40)$$

where B_v is a planted low-rank bilinear map. The target may read out selected internal nodes as well as the root. Ranks 1, 2, 4 provide a first complexity sweep.

6.3 T2: residual TTN teacher for soft hierarchy

To introduce bypass routes, use common node width r and

$$h_v^* = \rho(U_v h_{v_L}^* + V_v h_{v_R}^*) + \gamma B_v(h_{v_L}^*, h_{v_R}^*), \quad f^*(x) = c^{*\top} h_{\text{root}}^*. \quad (41)$$

The parameter ρ controls the strength of direct linear routes relative to multiplicative composition. Sweeping $\rho \in \{0, 0.25, 1\}$ with fixed γ creates a progression from hard to softer hierarchy. It does not guarantee causal gating; that must be tested by interventions.

6.4 T3: smooth DNN teacher

Train or sample a smooth DNN teacher and use its real-valued outputs or logits as labels. This removes exact polynomiality while preserving full access to the teacher’s hidden states and tangent kernel. The setup is harder to interpret because the relevant teacher features are not uniquely specified. Useful observables include output agreement, hidden-feature probes, CKA or subspace alignment, physical-cut singular modes, and the transfer of targeted hidden-unit or subspace interventions.

Two variants should be separated:

- a randomly initialized teacher with frozen weights, which gives a controlled random function;
- a teacher first trained on a structured task or dataset, which gives more realistic features but adds an additional training history.

6.5 T4: structured synthetic images and real images

Before MNIST, use rendered compositional images with known latent parts and a known spatial hierarchy. This preserves interpretable ground-truth features while introducing pixel-level nuisance variation.

For MNIST or Fashion-MNIST, begin with a fixed patch embedding and compare a modest MLP or residual MLP with a spatial TTN over the same patch blocks. Use one-vs-rest regression or soft-label distillation before full multiclass cross-entropy. Real images violate the independent-block assumption, so naive crossed recombinations of image halves may be off-distribution. Physical-cut analysis should therefore use either a conditional generative sampler, an explicitly empirical operator, or representation-space CCA rather than silently applying a product-measure interpretation.

6.6 Student arms

Every synthetic trial should include a common teacher and several student arms:

1. a smooth DNN student;
2. its exact checkpoint-dependent polynomial recurrence;
3. the exact tied computation TTN representation of that recurrence;
4. a free tied-core TTN initialized from the exact representation;
5. a physical-block TTN with ranks swept over a bracket;
6. a wrong-tree TTN and, when useful, a tensor train as structural controls.

The first three arms isolate polynomial approximation from implementation. The latter arms test the actual surrogate hypothesis.

6.7 What does “matched capacity” mean?

There is no unique scalar notion of equal capacity across a DNN and a TTN. The experiments should use a capacity bracket rather than one claimed match.

1. **Realizability check.** Verify that each student can fit the teacher to a predeclared low test error under a generous training budget. Underfit models are labeled as such rather than used to infer dynamical mismatch.
2. **Parameter and compute brackets.** Include TTN ranks below, near, and above the DNN parameter count and training cost.
3. **Initial scale match.** Match output variance and, in a separate condition, the initial function-space speed $\|\dot{f}(0)\|$ or tangent-kernel trace. These are predeclared calibrations, not post-hoc time warps.
4. **Approximation frontier.** Report fidelity as a function of rank and parameter count. A Pareto curve is more informative than a single matched point.

A TTN derived from the polynomialized DNN has the closest local learning capacity because it is the same function class under the same original parameters. A generic physical TTN may be more interpretable, but its capacity must be measured empirically.

7 Primary observables

7.1 Loss and function-space agreement

Record train, audit, and final-test losses. The primary model-to-model function distance is

$$d_{\text{fun}}(t) = \frac{\|f_{\theta(t)} - g_{\varphi(t)}\|_{L^2(\nu_x)}}{\max\{\|f^*\|_{L^2(\nu_x)}, \epsilon_{\text{scale}}\}}. \quad (42)$$

Also record output correlation and the residual component orthogonal to all named teacher features. Matching the teacher coefficients while producing a large spurious residual is not success.

7.2 Parameter-to-function map and tangent kernel

On a fixed audit set compare:

$$d_J = \frac{\|J_{\text{DNN}} - J_{\text{proxy}}\|_F}{\max\{\|J_{\text{DNN}}\|_F, \epsilon\}} \quad \text{when parameters are shared,} \quad (43)$$

$$d_K = \frac{\|K_{\text{DNN}} - K_{\text{TTN}}\|_F}{\max\{\|K_{\text{DNN}}\|_F, \epsilon\}}, \quad (44)$$

$$a_K(r) = \frac{1}{r} \|U_{K,\text{DNN}}^\top U_{K,\text{TTN}}\|_F^2, \quad (45)$$

where U_K contains the leading kernel eigenvectors. Also record the residual-weighted function-space vector-field defect from Eq. (36). Jacobian-vector products and randomized low-rank sketches can be used when full matrices are too large.

7.3 Named teacher-feature coefficients and acquisition times

For the staircase teacher, define

$$a_m^g(t) = \frac{\mathbb{E}[g_t(x)\phi_m(x)]}{\sqrt{\lambda_m}}. \quad (46)$$

The teacher has $a_m^{f^*} = 1$. Define acquisition times

$$\tau_m(a) = \inf\{t : a_m(t) \geq a\} \quad (47)$$

for $a \in \{0.1, 0.25, 0.5\}$, with a persistence rule for noisy measurements. If a crossing is not observed, report right-censoring.

For general TTN teachers, name selected node features and estimate their accessibility from student hidden states with held-out linear probes. Probe fit and probe test samples must be separate. Probe alignment is supporting evidence; it does not establish causal necessity.

7.4 Physical-cut hierarchical SVD

Let the actual input have blocks $x = (x^{(1)}, \dots, x^{(M)})$ with product law $\mu_1 \otimes \dots \otimes \mu_M$. For a cut $S \mid S^c$, a square-integrable function has a Schmidt decomposition

$$g(x_S, x_{S^c}) = \sum_{q \geq 1} s_{S,q}(g) \phi_{S,q}(x_S) \psi_{S,q}(x_{S^c}). \quad (48)$$

The singular values and resolved singular subspaces are architecture-independent output features. They should be computed on a predeclared physical dimension tree, not on arbitrary cuts of the repeated-input computation tree.

An empirical crossed-grid estimator draws $u_a \sim \mu_S$ and $v_b \sim \mu_{S^c}$ independently and forms

$$M_g[a, b] = \frac{g(u_a, v_b)}{\sqrt{n_L n_R}}. \quad (49)$$

Use the same grid for the teacher, DNN, and TTN. Report singular amplitudes, gaps, and subspace alignment

$$A_{S,r}(g, h) = \frac{1}{r} \left\| U_{S,r}(g)^\top U_{S,r}(h) \right\|_F^2. \quad (50)$$

Near a degeneracy, compare the whole singular subspace rather than arbitrarily labeled vectors. Estimate variability with independent grids or row-and-column resampling, not an i.i.d. bootstrap of matrix entries.

7.5 TTN-specific compression observables

On a physical tree record bond singular values, effective ranks, discarded singular mass, parameter count, wall time, and memory. If a compressed model is used dynamically, measure both

$$\delta_0 = \left\| g - g^{(r)} \right\|_{L^2}, \quad \delta_1 = \left\| \nabla_{\vartheta} g - \nabla_{\vartheta} g^{(r)} \right\|_{L^2}. \quad (51)$$

Snapshot function compression controls δ_0 but does not automatically control δ_1 .

7.6 Intervention response

Two interventions are primary.

Teacher-strength intervention. Reduce one prerequisite coefficient, for example $\lambda_2 \mapsto \lambda_2/4$, while holding data, directions, initialization, and optimizer fixed. Measure the downstream acquisition-time shift and compare it between DNN and TTN students.

Feature-blocking intervention. At a chosen checkpoint, identify a held-out probe direction for a named feature, whiten the relevant hidden state, and project out that direction. Apply the same fixed affine intervention to the DNN and its local polynomial proxy, rebuild the proxy for the intervened architecture, and compare immediate output shock separately from subsequent learning. Include identity, random-direction, and same-norm controls.

A surrogate has mechanistic fidelity only if intervention effects transfer, not merely if unperturbed learning curves look similar.

8 Experimental protocols

8.1 E0: algebra, gradients, and sharing

Use a tiny depth-two model with dimensions $(d_0, d_1, d_2) = (3, 4, 2)$ in float64 and quadratic Taylorization.

1. Compare the sequential polynomial recurrence, the tied-tree contraction, and an explicit degree-four coefficient contraction.
2. Compare values and gradients with respect to all original weights, biases, and the readout.
3. Repeat with an exactly quadratic activation; all centers must give the same polynomial.
4. Deliberately untie repeated occurrences or average rather than sum their gradients. These variants should fail the gradient-equivalence check.

Suggested tolerances are 10^{-10} relative prediction error and 10^{-8} relative gradient error, with a stated absolute floor near zero. This experiment must pass before any scientific comparison.

8.2 E1: checkpoint approximation and scaling laws

At each DNN checkpoint construct both fixed-center and example-centered proxies for $k \in \{1, 2, 3, 4\}$.

1. Measure zero-step prediction, Jacobian, loss-gradient, and tangent-kernel defects.
2. Perturb parameters along fixed random unit directions with radii $10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}$, restricted to a valid smooth region.
3. For the example-centered construction, test the expected upper-order behavior: prediction error of order $\|\Delta\|^{k+1}$ and Jacobian error of order $\|\Delta\|^k$.
4. For fixed centers, report the nonzero checkpoint error floor and correlate it with layerwise activation spread around the centers.

A fixed-center failure together with example-centered success indicates an input-wide polynomial approximation problem. The next global baselines should then include Chebyshev or distribution-weighted polynomial fits over the observed preactivation range, clearly labeled as non-Taylor surrogates.

8.3 E2: blind local rollouts

At checkpoint t_j , clone the same parameters and full optimizer state into the reference DNN continuation and each proxy. Freeze centers and activation coefficients throughout a forecast of H updates. Use identical training data and, for SGD, identical minibatch indices. The proxy may use only its own hidden states and may not load future reference parameters.

At forecast steps 0, 1, 5, 20, 100, 400 when available, record:

- train and test loss;
- function distance and parameter distance;
- common-point prediction, Jacobian, and loss-gradient defects;
- tangent-kernel distance and vector-field cosine;
- teacher-feature coefficients and acquisition-time forecasts;
- physical-cut singular values and resolved subspaces.

Define the accepted local horizon H^* as the largest step satisfying predeclared joint tolerances. A useful initial screen is normalized prediction RMS at most 0.02, maximum normalized named-feature error at most 0.05, and relative loss-gradient defect at most 0.10. These are empirical thresholds, not theoretical constants.

The primary analysis regresses H^* against Taylor order, checkpoint, depth, activation spread, gradient defect, and kernel defect. A proxy that matches loss but misses feature coefficients fails the feature-level objective.

8.4 E3: chart stitching and refresh cost

Compare three policies:

1. one fixed polynomial model built at initialization;
2. independent checkpoint-reset forecasts, each starting from the recorded true checkpoint;

3. carried-parameter forecasts, in which the proxy endpoint becomes the next initial state and centers are refreshed using the predicted parameters.

For carried forecasts, preserve optimizer memory. Report cumulative endpoint error, per-window growth, function and spectrum jumps at refresh, and the number of smooth-network evaluations required for construction and auditing. A method that repeatedly queries the reference gradient is oracle-assisted and must be reported separately from an autonomous simulator.

8.5 E4: global TTN versus DNN on a TTN teacher

Use the staircase teacher in Eq. (38), then the balanced bilinear teacher, then the residual teacher in Eq. (41). For every teacher seed, train the following paired students on the same data:

1. smooth MLP;
2. residual MLP;
3. physical TTN with the teacher tree and rank sweep;
4. wrong-tree TTN;
5. optional tensor-train control.

Primary comparisons use the raw optimizer time $t = \sum_n \eta_n$ with no post-hoc warping. A secondary diagnostic may plot trajectories against function-space arc length to distinguish a trivial speed difference from a different path, but this is not counted as evidence of literal dynamical equivalence.

Before interpreting dynamics, verify realizability and final-fit comparability. Then compare feature-trajectory RMSE, Spearman correlation of acquisition times, tangent-kernel subspace alignment, physical-cut HSVD, and intervention-effect transfer.

8.6 E5: relaxation path from the exact TTN to a free model

At selected DNN checkpoints initialize all TTN variants from the exact polynomial representation and run matched local forecasts:

1. original-parameter tied TTN;
2. free tied cores with the exact induced metric $D\Gamma D\Gamma^\top$;
3. block-diagonal metric approximation;
4. Euclidean free-core gradient;
5. untied core occurrences;
6. HSVD-truncated physical TTN;
7. wrong-tree physical TTN.

This experiment identifies whether fidelity is lost because of activation approximation, parameterization geometry, weight sharing, compression, or tree choice. It is the most direct test of whether the exact representation can be converted into a genuinely simpler model.

8.7 E6: DNN teacher

Use a small smooth teacher DNN with stored hidden states, logits, and kernel snapshots. Train a student DNN and several TTNs. Since named polynomial teacher features are unavailable, predeclare a set of teacher observables:

- centered logit trajectories and function distance;

- teacher hidden-state probe subspaces;
- teacher and student kernel eigenspaces;
- physical-cut output singular modes;
- responses to teacher hidden-subspace ablations.

Begin with regression on teacher outputs. Add label noise and classification only after the regression trajectories are understood.

8.8 E7: structured images, MNIST, and beyond

The realistic-data ladder is:

1. synthetic compositional images with known object parts;
2. one-vs-rest MNIST regression on fixed patches;
3. multiclass MNIST or Fashion-MNIST with soft labels;
4. a small residual image model and a spatial TTN on the same patch embedding;
5. only then, more complex natural images.

For every stage, retain a same-tree and wrong-tree condition, a rank sweep, and local polynomial proxies of the DNN. Because explicit coefficient tensors become infeasible, use sequential polynomial evaluation, Jacobian-vector products, kernel sketches, and sampling-based feature operators. Track the fraction of audit preactivations lying outside the fitted polynomial interval as a direct global-approximation failure indicator.

8.9 E8: causal and negative controls

Across synthetic experiments include:

- weakened prerequisite features and downstream clock coupling;
- direct feature-blocking and sham interventions;
- degree-matched non-nested targets;
- reversed teacher feature strengths;
- an incorrect physical tree;
- exact polynomial activations as a positive control;
- a diagonal or rank-one TTN when it is known to lack the intended cross-feature coupling.

These controls prevent ordinary spectral bias, target amplitude, parameter count, or tree alignment from being mistaken for hierarchical feature learning.

9 Concrete first study

The first study should be deliberately small enough that values, gradients, coefficient tensors, and HSVD diagnostics can all be audited.

Component	Initial configuration
Teacher	Staircase teacher with $M = 3$, block dimension $b = 4$, and $\lambda_m = 1/3$; repeat at $M = 4$.
DNN	Two hidden layers of width 32, SiLU activation, scalar affine output. Later vary $L \in \{2, 3, 4\}$ and width 64.
Global TTN	Teacher-aligned binary tree, leaf width $r_0 \in \{1, 2, 4\}$ and internal ranks swept to bracket the DNN parameter count; wrong-tree control.
Initialization	Independent Gaussian weights with standard deviation $s/\sqrt{d_{\text{in}}}$, zero biases, random output head; start with $s = 1$, then $s = 0.5$. Match initial output variance across arms.
Data	4096 train, 1024 center-calibration, 4096 audit, and 8192 final-test examples, all disjoint with fixed seeds.
Optimization	Full-batch gradient descent, no momentum, decay, noise, or normalization. Choose η on a separate pilot from $\{0.001, 0.003, 0.01, 0.03\}$.
Reference record	4400 updates; checkpoints at 0, 100, 500, 1000, 2000, 4000; ordinary metrics every 10 updates.
Local forecasts	Horizons $H \in \{1, 5, 20, 100, 400\}$ and orders $k \in \{1, 2, 3, 4\}$; centers frozen in each forecast.
Feature diagnostics	Staircase coefficients at every record; physical-cut crossed-grid SVD with 64×64 grid and a stability check at 128×128 .
Replication	Five paired seeds for screening; at least twenty paired seeds for confirmatory claims about acquisition order or intervention effects.

Table 3: Recommended first configuration. The values are defaults for a staged study, not theoretically optimal constants.

9.1 Minimal local-forecast algorithm

1. Train the reference DNN and save parameters plus full optimizer state at each checkpoint.
2. Keep future reference continuation inaccessible to proxy construction.
3. Record checkpoint preactivations on calibration and training inputs.
4. For each center policy and Taylor order, compute and detach centers and activation derivatives.
5. Clone parameters and optimizer state into the proxy.
6. Verify zero-step prediction and gradient defects.
7. For each horizon, restart from the identical cloned checkpoint and run the proxy using its own hidden states and the same data sequence.
8. Audit predictions, gradients, kernels, coefficients, and physical feature subspaces.
9. Evaluate old and refreshed charts at the same parameter value before attributing a spectrum change to learning.

9.2 Minimal global-comparison algorithm

1. Sample one teacher, data split, and paired initialization seed record.
2. Calibrate stable learning rates on separate pilot teachers and freeze them before main evaluation.
3. Verify that each student can reach the predeclared final-fit threshold.
4. Train all student arms on identical examples and record function-space and feature observables on a shared schedule.

5. Run teacher-strength and feature-blocking interventions with all nonintervened quantities fixed.
6. Aggregate paired differences across seeds and report fidelity-versus-rank Pareto curves.

10 Statistical analysis and decision criteria

10.1 Primary endpoints

Local endpoint. The accepted forecast horizon H^* under joint prediction, feature, and gradient tolerances.

Global endpoints. Feature-trajectory RMSE, acquisition-time rank correlation, physical-cut subspace alignment, kernel alignment, and the difference between DNN and TTN intervention effects.

Reduction endpoint. The smallest physical TTN rank profile that preserves function, Jacobian or kernel, feature, and intervention tolerances.

10.2 Paired design and uncertainty

Use the same teacher, data, and initialization seed wherever a pairing is meaningful. Report paired seed differences and confidence intervals rather than only separate confidence bands. Five seeds are an exploratory screen; confirmatory claims need stronger replication. Use independent final-test samples and independent SVD grids. For crossed-grid matrices, resample rows and columns or repeat the grid; do not treat all entries as independent.

When many cuts and feature thresholds are examined, predeclare a primary subset and treat the remainder as exploratory. Report unresolved singular gaps and censored acquisition times explicitly.

10.3 Success criteria

A TTN surrogate is scientifically useful only if it passes the relevant level:

Algebraic fidelity.

The TTN and polynomial recurrence agree to numerical precision in values and original-parameter gradients.

Local dynamical fidelity.

The proxy predicts a nontrivial withheld DNN continuation without using future parameters, with gradient and feature agreement as well as loss agreement.

Global feature fidelity.

Independently trained DNN and TTN students acquire the same named or physical-cut features in comparable order across seeds.

Mechanistic fidelity.

Interventions on prerequisite strength or accessible feature directions have comparable downstream effects.

Reduction.

A materially smaller set of TTN ranks or modes preserves the preceding predictions.

If only the exact full-width tied representation succeeds, the result is a controlled representation but not yet a reduced theory.

11 Failure modes and breakdown diagnostics

Failure signature	Likely interpretation	Diagnostic or next experiment
Fixed-center proxy fails at step zero; example-centered proxy succeeds	Activation spread across examples, not failure of the tensor identity	Higher order; multiple input patches; Chebyshev or distributional fit; report spread and tail mass
Predictions close but gradient or kernel far	Wrong local geometry of the parameter-to-function map	Compare Jacobians, induced core metric, and residual-weighted vector field
Exact tied proxy works; Euclidean free-core TTN fails	Parameterization geometry is essential	Test induced, block-diagonal, and natural-gradient approximations
Full proxy works; compressed TTN fails dynamically	Compression preserves snapshots but not derivatives	Measure δ_1 ; use differentiable low-rank parameterization or projected vector field
Teacher-aligned tree works; wrong tree needs large ranks	Data hierarchy is real and tree-specific	Rank-versus-error curves; tree search only after fixed-tree controls
All trees require rapidly growing ranks	Weak tensor compressibility or mischosen input features	Change leaf feature map; inspect physical-cut spectra; accept negative result
Good loss agreement, wrong named features	Surrogate captures task difficulty but not feature dynamics	Feature coefficients, residual output, and intervention tests are decisive
Large jumps at chart refresh	Representation change confused with learning	Evaluate old and new charts at identical parameters; separate chart and parameter updates
Real-image crossed-grid modes unstable	Product-measure estimator is off-distribution	Conditional sampling, empirical operators, or representation-space alternatives

Table 4: A diagnostic taxonomy for surrogate breakdown.

Additional stress tests should vary depth, width, activation saturation, input tails, label noise, minibatch size, momentum, Adam-like optimizer state, residual strength, and normalization. Each complication should be introduced only after the preceding simpler stage is understood.

12 Expected outcomes and theory roadmap

Several outcomes would all be informative.

Strong positive outcome. Low-order local TTNs predict feature trajectories for substantial windows, and low physical ranks preserve both functions and kernels. Independent TTNs reproduce the DNN’s feature order and intervention response on hierarchical teachers. This would motivate reduced evolution equations for selected coefficient modes, singular subspaces, and tangent-kernel couplings.

Local-only outcome. Checkpoint-dependent TTNs forecast DNN training, but no one autonomous TTN works globally. The appropriate theory would be a charted model with controlled transition errors rather than a single global surrogate.

Representation-only outcome. The exact tied TTN works, but relaxing constraints or compressing ranks destroys fidelity. The tensor language remains a useful algebraic description, but not yet a simpler dynamical model. The next theoretical target would be the induced metric in Eq. (27), not free-core gradient flow.

Negative outcome with structured diagnosis. Low-order polynomialization fails because pre-activation ranges are too broad or physical ranks grow too quickly. This would delimit the regimes in which TTNs are plausible surrogates and motivate piecewise-polynomial, distribution-adapted, or non-tree tensor models.

The first theoretical milestone after successful experiments is an approximate closed equation for a small observable set. On a fixed polynomial chart with coefficient vector $C(\theta)$ in an orthonormal basis,

$$\dot{C} = -D_\theta C D_\theta C^\top (C - C^*) \quad (52)$$

for population square loss. For a simple singular value s_q of a cut matrix $M(C)$,

$$\dot{s}_q = u_q^\top \dot{M} v_q. \quad (53)$$

These identities are not closed because the coefficient Jacobian and singular vectors evolve. The empirical objective is to discover regimes in which only a small subset of their couplings matters.

13 Reproducibility checklist

Every run should store:

- the full teacher graph, weights, normalization constants, and data seeds or hashes;
- exact activation formulas and derivative implementation;
- initial parameters, optimizer state, learning-rate schedule, and checkpoint indices;
- center policy, frozen centers, derivative coefficients, Taylor order, and polynomial fitting interval;
- whether a forecast is checkpoint-reset, carried, or oracle-assisted;
- whether compression is used only for measurement or also for training;
- absolute and normalized prediction, gradient, kernel, feature, and subspace errors;
- activation ranges, out-of-range fractions, singular gaps, and rank profiles;
- wall time, memory, and smooth-network queries used for refresh or audit.

In automatic differentiation code, centers and activation derivatives must be detached from the live graph, while the tensor cores recomputed from current neural weights must remain differentiable. Use the efficient polynomial recurrence for training and reserve explicit large tensor contractions for equivalence checks and small spectral studies.

14 Conclusion

The polynomial approximation of a smooth depth- L neural network can be represented exactly as a constrained, tied, k -ary TTN on repeated inputs. This establishes an algebraic bridge but not, by itself, a dynamical or interpretive one. The proposed research program therefore compares models at four levels: functions, parameter-to-function geometry, hierarchical features, and causal response. It

begins with exact polynomial controls and TTN teachers, then progressively relaxes core constraints, compresses physical ranks, introduces residual routes, replaces the teacher by a DNN, and finally moves to images. At each stage, the design identifies the first point at which the surrogate breaks.

A successful result need not be one global TTN that reproduces every detail of DNN training. A sequence of low-order local tensor models that accurately predicts feature acquisition and interventions over useful windows would already provide a principled, testable intermediate theory. The decisive criterion is not representability alone, but whether a small tensor description retains the training dynamics that matter.

References

- [1] *Local Polynomial Tensor Networks for Deep Neural Network Training: Exact Tree Representations, Controlled Changes of Local Model, and a Teacher-Student Experimental Protocol*. Hierarchical Feature Learning Project research note, 1 October 2026.
- [2] N. Cohen, O. Sharir, and A. Shashua. On the expressive power of deep learning: A tensor analysis. *Proceedings of COLT*, 2016.
- [3] L. Grasedyck. Hierarchical singular value decomposition of tensors. *SIAM Journal on Matrix Analysis and Applications*, 31(4):2029–2054, 2010.
- [4] P. Morala, J. A. Cifuentes, R. E. Lillo, and I. Ucar. NN2Poly: A polynomial representation for deep feed-forward artificial neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36(1):781–795, 2025.
- [5] Y. Bai, B. Krause, H. Wang, C. Xiong, and R. Socher. Taylorized training: Towards better approximation of neural network training at finite width. arXiv:2002.04010, 2020.
- [6] J. Huang and H.-T. Yau. Dynamics of deep neural networks and neural tangent hierarchy. *Proceedings of ICML*, 2020.